

# Codelt Sprint AI 14기

## 초급 프로젝트

4팀

팀장: 신유정

팀원: 김건, 김예원, 원재민, 이주엽

### 1. 프로젝트 개요

#### 1.1. 프로젝트 배경 및 목표

헬스케어 스타트업의 AI 엔지니어링 팀으로서 유저가 본인의 모바일 애플리케이션으로 자신이 복용중인 약 사진을 찍었을 때, 이미지 인식을 통해 해당 약에 대한 정보를 확인할 수 있는 모델을 만들어야하는 미션을 부여받았다고 가정한다. 프로젝트의 목표는 사진 속에 있는 최대 4개의 알약의 이름(클래스)과 위치(바운딩 박스)를 검출하는 모델을 구현하고, 성능을 지속적으로 개선해나가는 것이다.

#### 1.2. 팀원 역할 분담

- 신유정: Github 관리, 리팩토링, RF-DETR, 앙상블
- 김건: Faster R-CNN, 성능 고도화, 회의록 작성, 중간 발표자료 취합
- 김예원: RetinaNet, 최종 발표자료 작성
- 원재민: YOLO, 모델 학습 관련 자료 작성
- 이주엽: YOLO, 데이터 관련 자료 작성

### 2. 학습 데이터셋 구축 및 분석

#### 2-1. 초기 데이터셋

기본 제공 데이터셋은 경구약제 이미지 기반의 알약 탐지를 목표로 구성된 초기 훈련 및 검증용 데이터입니다.

| 항목     | 내용 및 상세 수치                                                | 비고                                                            |
|--------|-----------------------------------------------------------|---------------------------------------------------------------|
| 데이터 출처 | 경구약제 알약 탐지 경진대회 기본 제공 데이터셋 (AI Hub 경구약제 이미지 데이터를 가공하여 제공) | 폴더 구성:<br>train_images/<br>train_annotations/<br>test_images/ |
| 이미지 수  | 초기 제공 이미지 수 (Train: 232 / Test: 842)                      | 원본 데이터                                                        |

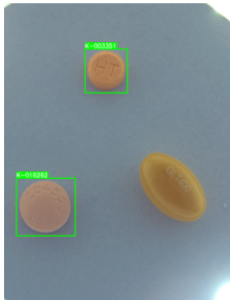
|         |                                        |              |
|---------|----------------------------------------|--------------|
| 클래스 수   | 56개 클래스 (알약 종류)                        | 초기 클래스 범주    |
| 저장 포맷   | .png 이미지 파일 + .json 라벨 파일 구조           | COCO JSON 구조 |
| BBox 포맷 | 절대 픽셀 좌표계 ( $x_{min}, y_{min}, w, h$ ) | COCO Format  |
| 이미지 해상도 | 976 × 1280 (전체 이미지 동일)                 | 고정 해상도       |

## 2-2. 초기 데이터 EDA 및 품질 점검

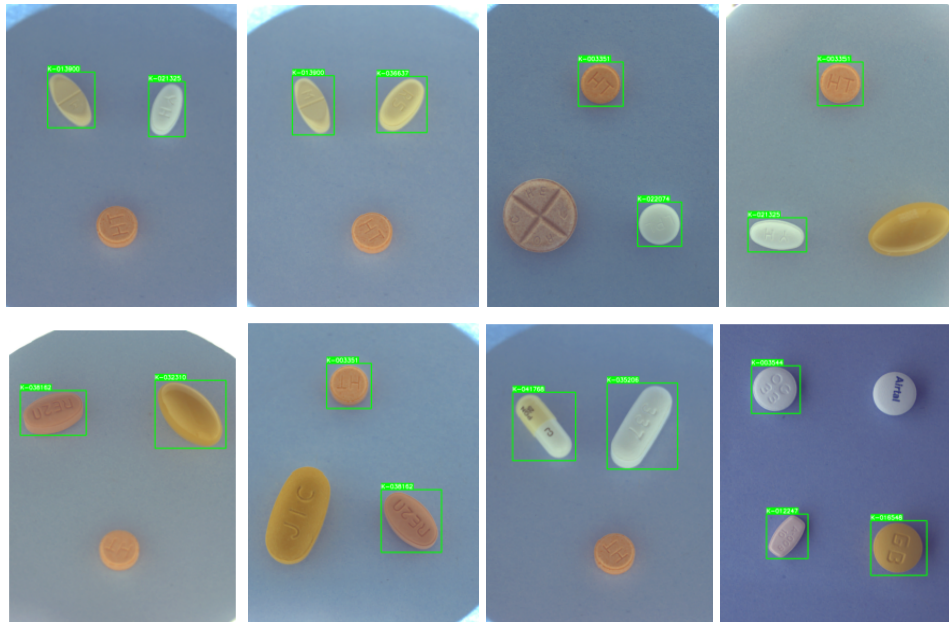
### ① 라벨 데이터 품질 결함 (Data Quality Issues)

원본 라벨 데이터셋에 대한 전수 검증 스크립트를 실행한 결과, 모델 학습 오작동을 유발할 수 있는 아래 3가지 유형의 라벨 결함을 발견하여 정제(Data Cleaning)를 진행하였습니다.

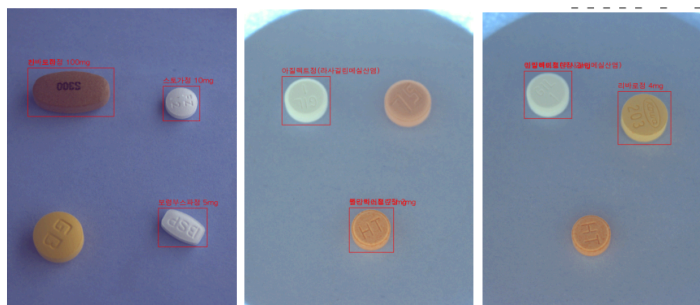
- 경계 초과 BBox (1건): Bounding Box의 좌표 값이 원본 이미지의 해상도 범위(976 × 1280)를 벗어난 예외 건



- Annotation 라벨 누락 (8건): 이미지 파일은 존재하나 대응하는 정답 라벨 파일이 누락된 건

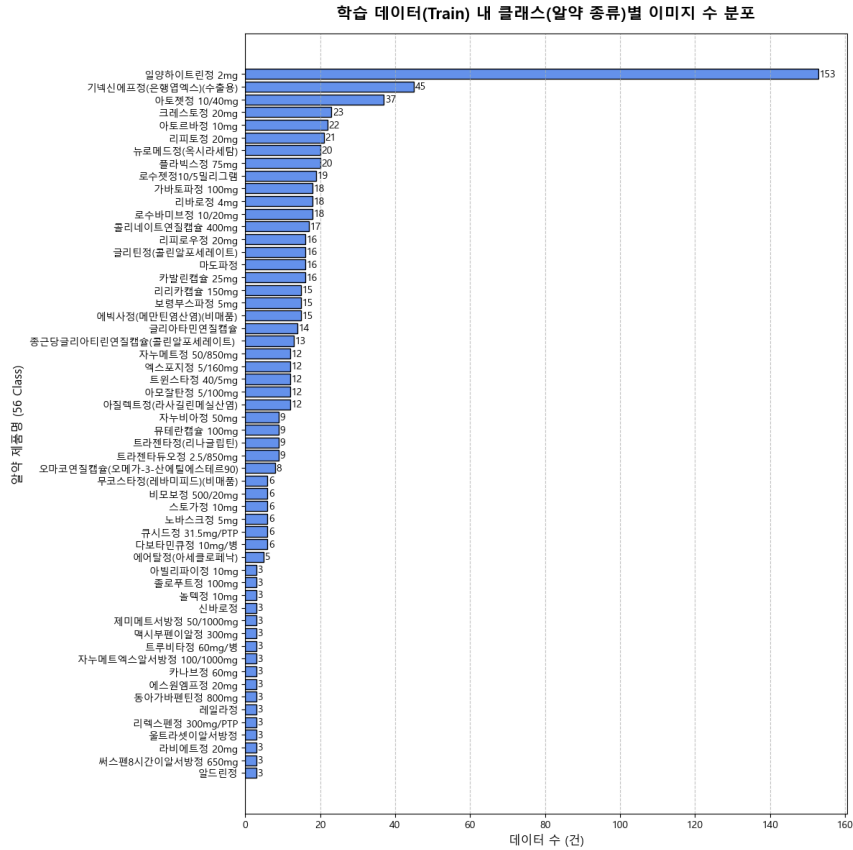


- **중복 BBox 좌표 (3건):** 2개 이상의 Bounding Box가 동일한 위치를 중복으로 가리키고 있는 건



## ② 클래스 분포 및 불균형 (Class Imbalance)

- 클래스별 데이터 수량이 최소 3개에서 최대 157개까지 극심한 데이터 불균형(Class Imbalance) 현상을 보임
- 특정 희귀 알약 클래스의 학습 데이터 부족으로 인해 모델의 일반화 성능 저하가 우려되어, 외부 데이터셋(AI Hub)을 활용한 클래스 보강 필요성을 확인  
- 클래스 별 데이터 수 분포



## 2-3. 형식 변환 및 라벨 포맷 재정의

모델별 학습 파이프라인에 맞춰 **COCO** 포맷 라벨 데이터를 표준 포맷으로 변환하고, 파이프라인 검증 과정에서 발견된 **BBox** 수식 오류를 보정하였습니다.

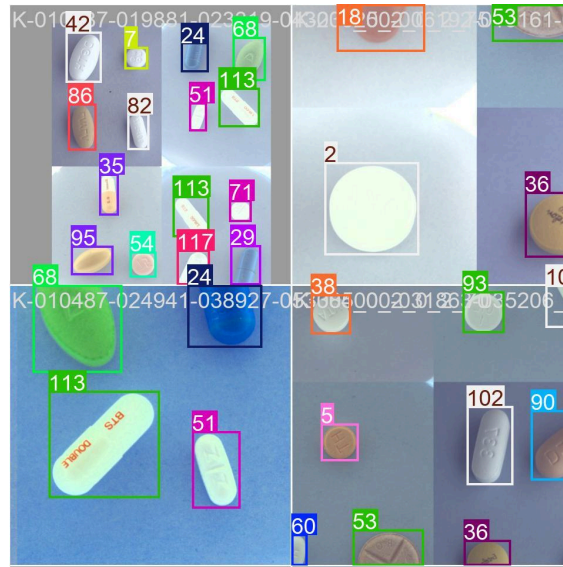
### ① 모델별 라벨 포맷 변환

- **YOLO 포맷:**
  - 이미지당 1개의 .txt 파일로 라벨 병합
  - 좌표 정규화 (0.0 ~ 1.0) 및 0-based Class Index (0 ~ 56) 변환
  - 정규화 중심 좌표 수식 적용:  $x_{center} = \frac{x_{min}+w/2}{W}$ ,  $y_{center} = \frac{y_{min}+h/2}{H}$
- **Faster R-CNN 포맷:**
  - 원본 COCO 포맷의 좌표 그대로 이용
  - 별도의 라벨 매핑 절차 없이 이용
- **RF-DETR 포맷:**
  - Roboflow RF-DETR 입력 규격에 맞춘 폴더 구조
  - COCO JSON과 라벨 내용은 동일

### ② BBox 좌표 정규화 수식 보정 (Troubleshooting)

- 증상: 초기 검증 과정에서 Validation Recall이 2.2% ~ 8.5% 수준으로 극단적으로 낮게 산출되었으며, 시각화 시 BBox가 좌상단(Top-Left) 구석으로 찌그러지는 현상 포착

- 원인 및 해결: 중심 좌표 계산 시  $x_{min}$ 에  $w/2$  대신  $w$  전체를 더했던 수식 오류를 발견. 수식 보정 후 학습 배치 시각화(train\_batch)를 통해 정답 BBox가 알약 중앙 위치에 정확하게 얹히는 것을 확인  
- 수식 보정 후 학습이미지 일부



## 2-4. 외부 데이터 통합 (AI Hub 경구약제 데이터셋)

### ① 도입 배경 및 스키마 맞춤 (Schema Standardization)

- 도입 목적: 훈련 데이터셋에 존재하지 않지만, 평가 데이터셋에 존재하는 알약 클래스 데이터를 보강하고, 수량이 적은 희귀 클래스의 일반화 성능 향상
- 포맷 통합: AI Hub 라벨 파일(JSON)을 파싱하여 기존 프로젝트 데이터셋 규격에 맞춰 카테고리 ID 및 좌표 체계 통합
- Label Map 확장: 데이터 증강에 따라 라벨 맵 재정의의 통해 전체 클래스 범주를 기존 56개에서 118개로 확장

### ② 결함 데이터 정제 전략

- 1차 전처리 시 구축한 이상치 검출 알고리즘을 AI Hub 데이터셋에도 동일하게 적용
- 좌표 오차가 존재하는 결함 데이터는 AI Hub 데이터 도입으로 전체 데이터가 충분히 확보되었다고 판단하여 전량 삭제

## 2-5. 최종 데이터셋

| 구분<br>항목 | 1차<br>기본<br>데이<br>터셋 | 2차 최종 데이터셋<br>(AI Hub 통합) | 비고 |
|----------|----------------------|---------------------------|----|
|          |                      |                           |    |

|                            |          |           |                              |
|----------------------------|----------|-----------|------------------------------|
| 전체<br>이미지<br>수             | 220<br>장 | 10,405 장  | Train 7,954 / Val 2,451      |
| 전체<br>클래스<br>수             | 57 개     | 118 개     | +61개 신규 클래스 확장               |
| Train /<br>Val<br>분할<br>비율 | 8 : 2    | 8 : 2     | -                            |
| 최종<br>학습<br>설정<br>파일       | -        | data.yaml | 경로, nc, Label Map<br>딕셔너리 정의 |

- data.yaml

```
data > augmented > processed > ! data.yaml
1  path: data/augmented/processed
2  train: train/images
3  val: val/images
4  nc: 118
5  names:
6      0: K-000250
7      1: K-000573
8      2: K-001866
9      3: K-001900
10     4: K-002483
11     5: K-003351
12     6: K-003483
13     7: K-003544
14     8: K-003614
15     9: K-003743
16     10: K-003832
```

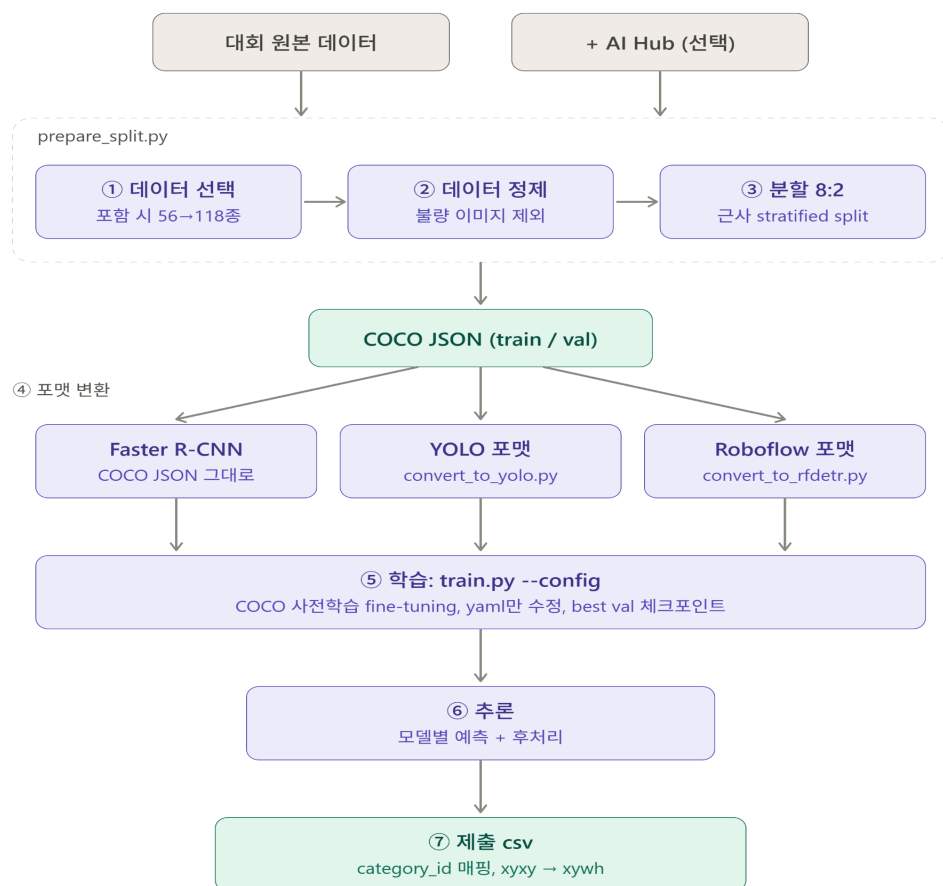
### 3. 모델링

#### 3.1. 비교 모델 선정

본 프로젝트는 모바일 어플리케이션에 들어갈 이미지에 포함된 알약의 종류와 위치를 검출하는 모델을 설계하는 것을 목표로 한다. 알약 종류를 잘못 추론할 경우 사용자의 건강에 영향을 줄 수 있기 때문에 정확도가 중요하다. 사용자의 편의성을 위해 추론 시간이 짧을 수록 좋다. 이러한 목표를 위해 모델 별 정확도와 추론 비용을 비교하기 위해 **Faster R-CNN**, **RetinaNet**, **YOLO**, **RF-DETR** 네 모델을 후보로 선정하였다. **Faster R-CNN**은 **2-stage** 모델로 속도는 느리지만 정확도가 높기 때문에 프로젝트 내의 다른 **1-stage** 모델들과 속도-정확도 비교군으로 선정하였다. **RetinaNet**은 **1-stage** 구조로 빠른 객체 탐지가 가능하고, **Focal Loss**를 통해 클래스

불균형을 완화할 수 있으며, FPN을 통해 다양한 크기의 객체를 효과적으로 탐지할 수 있는 장점이 있다. YOLO는 1-stage 객체 탐지 모델로, 별도의 영역 제안 단계 없이 이미지에서 객체의 위치와 클래스를 한 번에 예측하여 빠른 추론 속도를 갖는 것이 특징이다. 또한 다양한 크기의 경량 모델을 제공하여 모바일 환경과 같이 추론 속도와 연산 비용이 중요한 환경에 적용하기 용이하다. 높은 정확도와 빠른 추론 속도의 균형을 갖추고 있어 본 프로젝트의 실제 서비스 적용 가능성을 고려한 모델로 선정하였다. RF-DETR는 Transformer기반의 모델로 비슷한 속도의 다른 Transformer 모델보다 정확도가 높다. CNN 기반의 모델과 앙상블시 성능에 효과를 볼 수 있을 것이라는 기대에 선정하였다. 본 비교의 목적은 동일한 평가 데이터에서 검출 성능과 추론의 속도의 차이를 확인하고 최종 모델을 선정하기 위한 것이다.

### 3.2.모델링 적용 방식



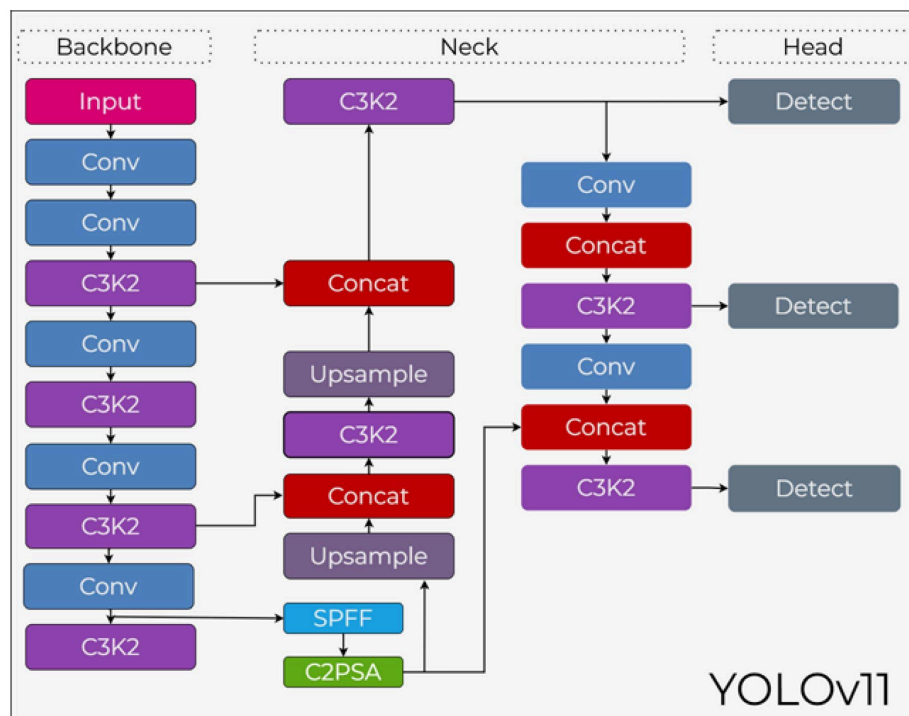
1. 데이터 선택: 대회 원본에 AI Hub 추가 데이터를 넣을지 prepare\_split.py 옵션으로 선택 (넣으면 56종 → 118종)
2. 데이터 정제: bbox 형식 오류, 중복 박스, 이미지 밖 박스, 파일명의 알약 수와 박스 수 불일치 등 불량 이미지 제외 (AI Hub 포함 시 330장)
3. 분할: train/val 8:2, 모든 클래스가 val에 들어가도록 클래스 비율을 맞춘 근사 stratified split. 결과는 COCO JSON으로 저장
4. 포맷 변환: Faster R-CNN은 COCO JSON을 그대로 쓰고, YOLO와 RF-DETR용으로는 각각 YOLO 포맷과 Roboflow 포맷으로 변환

5. 학습: 세 모델 모두 COCO 사전학습 가중치에서 fine-tuning하고 val 성능이 가장 좋은 체크포인트 사용, 모델과 하이퍼파라미터는 yaml 설정 파일로 관리해 같은 명령어(train.py -config)로 config 파일만 수정해 학습
6. 추론: 모델 별로 예측하고 후처리
7. 제출: 클래스 번호를 대회 category\_id로 매핑하고, 박스를 xyxy → xywh로 변환해서 csv로 저장

### 3.3. YOLO

YOLO는 입력 이미지를 한 번에 처리하여 객체의 위치(Bounding Box)와 클래스(Class)를 동시에 예측하는 1-stage 객체 탐지 모델이다. 별도의 Region Proposal 과정을 거치지 않고 한 번의 네트워크 연산을 통해 객체를 탐지하기 때문에 2-stage 객체 탐지 모델에 비해 빠른 추론이 가능하다. 본 프로젝트에 사용한 YOLO11n은 이전 YOLO 계열의 구조를 기반으로 정확도와 연산 효율을 개선한 모델이며, 다양한 크기의 모델을 제공하여 사용 환경에 따라 모델의 성능과 추론 비용을 조절할 수 있다. YOLO11n은 Nano 모델로 상대적으로 적은 파라미터와 연산량을 가지기 때문에 추론 속도와 모델 크기가 중요한 모바일 애플리케이션 환경에 적합한 모델 후보로 선정하였다. 이후 동일한 평가 데이터에서 탐지 성능과 추론 속도를 측정하여 다른 객체 탐지 모델과 비교하였다.

YOLO11의 전체 구조는 크게 Backbone, Neck, Detection Head로 구성된다.



Backbone에서는 입력 이미지로부터 객체의 특징을 추출한다. YOLO11에서는 기존 YOLO 계열의 효율적인 특징 추출 구조를 개선하여 적은 연산량으로 의미 있는 특징을 추출하도록 구성되어 있다. 또한 C2f 계열의 구조를 개선한 C3k2 블록을 사용하여 특징 추출 과정의 효율성과 표현력을 높였다.

Neck에서는 Backbone에서 추출된 서로 다른 해상도의 Feature Map을 결합한다. 이를 통해 고해상도 Feature Map의 세부적인 위치 정보와 저해상도 Feature Map의 의미 정보를 함께 활용할 수 있으며, 크기가 서로 다른 객체를 탐지하는 데 도움을 준다. 특히 알약 이미지와 같이 하나의 이미지에 여러 크기의 객체가 존재하는 경우 다양한 Scale의 특징을 활용할 수 있다는 장점이 있다.

Detection Head에서는 추출된 특징을 바탕으로 객체의 위치와 클래스를 예측한다. YOLO11 역시 Anchor-free 방식을 사용하여 사전에 정의된 Anchor Box에 의존하지 않고 객체의 위치를 직접 예측한다. 또한 객체의 위치를 나타내는 Bounding Box와 객체의 종류를 판단하는 Classification을 분리하여 처리하는 구조를 사용한다.

학습 데이터는 기존의 COCO 형식의 [x, y, width, height] Bounding Box 좌표를 YOLO 형식인 [class\_id, x\_center, y\_center, width, height]로 변환하여 학습 데이터셋을 구성하였다. Bounding Box 좌표는 이미지 크기에 맞게 0~1 범위로 정규화 하였다.

## 4. 모델 선정 실험 및 결과

### 4.1. 모델 별 초기 성능 비교 (원본 데이터만 학습)

| 모델           | 파라미터<br>(epoch/batch/lr) | Val<br>mAP50 | Val<br>mAP50-95 | Val<br>mAP75 | Kaggle  | 평균<br>추론<br>속도<br>(ms/img) |
|--------------|--------------------------|--------------|-----------------|--------------|---------|----------------------------|
| Faster R-CNN | 100/8/0.001              | 1.0000       | 0.9248          | 1.0000       | 0/32503 | 4980.84                    |
| RetinaNet    | 100/8/0.001              | 0.9762       | 0.9630          | 0.9762       | 0.37390 | 1530.06                    |
| YOLOv8s      | 100/8/0.001              | 0.9905       | 0.9855          | 0.9850       | 0.39262 | 315.77                     |
| RF-DETR      | 50/8/9.0001              | 0.9980       | 0.9841          | 0.9925       | 0.39434 | 329.55                     |

### 4.2. Alhub 데이터 추가의 효과

| 모델           | YOLO  |                       |
|--------------|-------|-----------------------|
| 클래스 수        | 56    | 118(원본 56 + Alhub 62) |
| Epochs       | 50    | 30                    |
| Batch        | 16    | 16                    |
| LR           | 0.001 | 0.001                 |
| Kaggle score | 0.346 | 0.611                 |

### 4.3. 앙상블 (YOLO + RF-DETR) 실험

YOLO11n과 RF-DETR Nano의 예측을 WBF(Weighted Boxes Fusion)로 합쳤다. NMS는 겹치는 박스 중 하나만 남기지만, WBF는 두 모델의 박스 좌표를 점수로 가중 평균해 더 정확한 박스를 만든다. RF-DETR Nano의 경우 Epoch 20까지 학습을

하였지만 **Epoch 10**의 결과보다 성능이 낮게 나와 과적합 된 것으로 판단하여 **Epoch 10**의 **weight**를 사용하였다.

모델 가중치: YOLO 1.0 : RF-DETR 1.0

iou\_thr: 0.5 (이상 겹치는 박스는 합침)

skip\_box\_thr: 0.0001

최대 예측 수: 제한 없음

| 모델           | YOLO11n | RF-DETR Nano | YOLO11n + RF-DETR Nano |
|--------------|---------|--------------|------------------------|
| 클래스 수        | 118     | 118          | 118                    |
| Epochs       | 30      | 10           | -                      |
| Batch        | 16      | 8            | -                      |
| LR           | 0.001   | 0.0001       | -                      |
| mAP50-95     | 0.9778  | 0.9835       | -                      |
| Kaggle score | 0.61139 | 0.61164      | 0.60619                |

양상불 점수는 단독 모델보다 낮았다. CNN과 Transformer 기반 모델을 양상불 시 서로 보완해 줄 것이라고 예상했지만 예측 결과인 csv파일을 비교해봤을때 모든 박스를 같은 위치에서(IoU 0.94이상) 같은 클래스로 매칭하였고, 한 모델이 놓친 알약을 다른 모델이 잡아주는 경우가 없었다.

#### 4.4. 최종 모델 선정

최종 모델은 YOLO11n으로 선택했다. Faster R-CNN과 RetinaNet은 추론 속도가 확연하게 느렸고, 양상불의 경우 두 단독 모델보다 점수가 낮았다. RF-DETR의 경우 YOLO 모델과 비슷한 점수가 나왔지만 체크포인트 크기가 YOLO11n 5.6MB, RF-DETR 122MB로 약 20배 차이가 난다. 실제 서비스에서는 속도와 배포 부담이 중요하다고 판단해 YOLO를 선택했다.

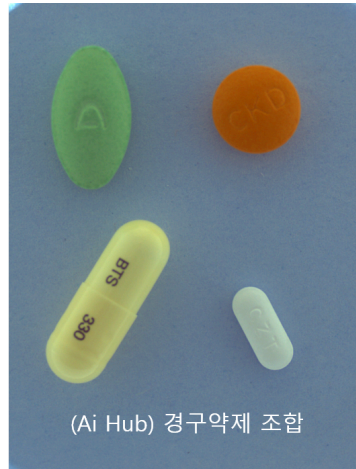
## 5. 실험 및 개선활동

### 5.1.외부 데이터 활용

#### 5.1.1.외부 데이터 가져오기

이번 대회에서 제공하고 있는 학습 데이터와 평가 데이터는 각각 232장과 842장이었다. 초기 모델링 과정에서 평가 데이터에 학습 데이터에 포함되어 있지 않은 알약의 존재를 확인하였으며, 이는 평가 데이터에는 학습 데이터에서 확인한 58종의 클래스 이외에 더 많은 클래스가 있다는 것으로 판단했다. 이를 보완하기 위하여 Kaggle 내 data 안내에 따라 외부 데이터를 가져오기로 판단하여, 'Ai Hub 경구약제 이미지 데이터'를 분석하였다. 해당 데이터 베이스는 크게 '경구약제조합'과 '단일경구약제'로 분류되어 있었으며, 일부를 내려받아 육안으로 평가한 결과는 그림 5-1과 같다. 시간과 자원의 한계로 모든 데이터를 이용할 수

없어 프로젝트에서 제공된 데이터와 유사도가 높은 ‘경구약제조합’만을 선택하였다. 그 결과 기존의 56종이었던 클래스가 118종으로 확장되었다.



[그림 5-1] 프로젝트 기본 제공 이미지와 Ai Hub 확장 이미지

두 데이터셋(기본 제공 데이터, Ai Hub 데이터)는 같은 이미지를 ‘70’, ‘75’, ‘90’이라는 이름에 차이만 두고 알약의 배치와 구성이 유사하였다. 총 이미지 데이터의 수가 10,689건에 도달하여 이를 모두 사용하면 학습 데이터의 양이 많아 결과가 좋아질 수 있으나, 시간적 자원적 제약이 있었으며, [그림 5-2]과 같이 ‘70’과 ‘75’는 거의 동일한 사진으로 판단하여 중복 이미지 데이터를 처리하는 기준을 설정하였다. 경계 상자가 3개 미만인 경우 라벨의 누락으로 판단하여 제외시켜 40건의 이미지를 제외 시켰으며, 동일 조합(분류 이름까지 동일한 조합) 내에서는 다음 우선순위로 선택했다.



[그림 5-2] ‘70’-‘75’-‘90’별 이미지 구성

- ① ‘70’ + ‘90’
- ② ‘70’이 없을 경우 ‘75’-‘90’

- ③ '90'이 없을 경우 '70'-'75'  
 ④ '75'

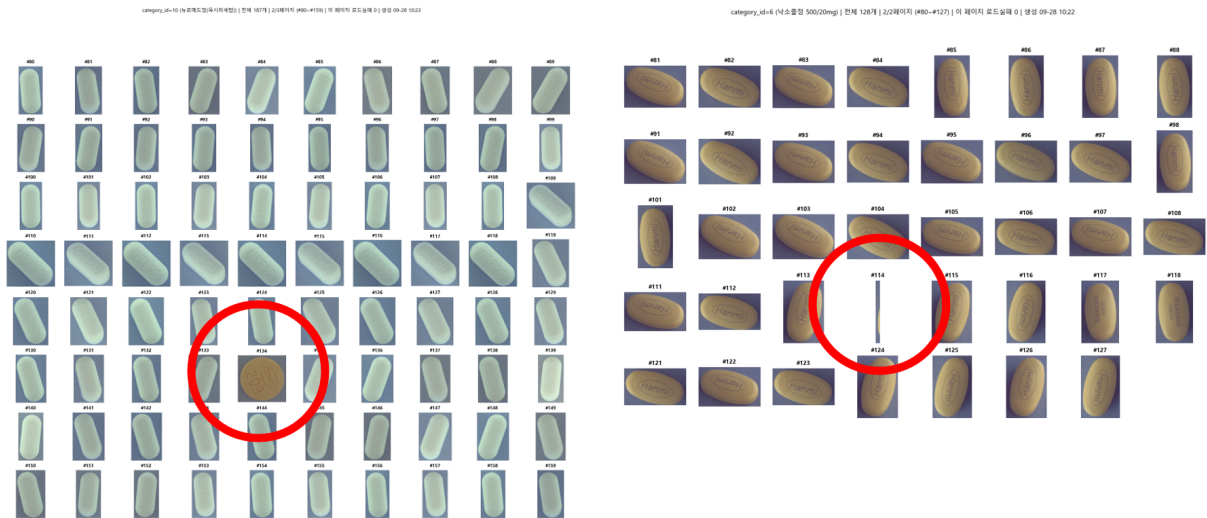
이러한 결정은 육안으로 비교하였을 때 '70'과 '90' 촬영본이 알약의 배치와 형태를 가장 뚜렷하게 보여주는 반면, '75'와 '90'은 서로 유사하여 다양한 이미지를 확보하는 의미가 부족하다고 판단했기 때문이다. 외부 데이터 가져오기 및 통합과정별 데이터 수의 변화는 [표 5-1]과 같다.

[표 5-1] 데이터 도입 과정의 이미지 및 클래스 변화

| 조치 사항              | 학습 이미지 |         | 클래스   |     |
|--------------------|--------|---------|-------|-----|
|                    | 수량 (장) | 증감      | 수 (개) | 증감  |
| 기본 제공 데이터          | 232    | +232    | 56    | +56 |
| 경계상자 3개/장 미만 관련 삭제 | 192    | -40     | 56    | -   |
| Ai Hub 데이터 가져오기    | 10,689 | +10,497 | 118   | +62 |
| 동일 조합 중복 제거        | 7,147  | -3,542  | 118   | -   |

#### 5.1.2. 이미지-클래스 매칭 오류 검토

불완전한 데이터라는 의견과 코칭에 따라 조금 더 면밀하게 데이터를 재확인하였다. 동일한 클래스로 라벨링된 경계 상자를 모아 그리드 형태로 시각화한 결과([그림 5-3]), 같은 클래스 안에 다른 알약이 포함되어 있음을 확인했다. 이는 외부 데이터의 라벨링 과정에서의 오류나 업로드 이후 추가적인 데이터 정제과정이 없기 때문에 발생한 것이라 추정한다. 우리 팀은 이러한 데이터 사이의 작은 오류도 학습 성능 저하에 영향을 미칠 것으로 판단하고 118종 모든 클래스를 확인하는 전수검사를 실시하였다. 검토 결과 잘못된 클래스에 배치된 건은 총 94건으로 판단하여 재배치하였으며, Bbox가 너무 작아 학습에 어려움이 있을 것으로 판단한 경우는 53건이 있었고, 이를 삭제하였다. 상기 조정은 해당 알약이 속한 사진 전체를 제거하거나 변경한 것이 아니라 개별 경계상자 단위로 적용하였다.



[그림 5-3] 이미지와 클래스 미스매칭 및 Bbox 오류 예시

## 5.2. 하이퍼-파라미터 튜닝

### 5.2.1. 대표 모델(대조군) 선발 및 튜닝 설계

기본 모델 선발은 09/28 기준으로 Kaggle Score가 가장 우수한 “YOLO v11n” 모델을 선발하였다. 팀원들은 각각 담당 모델을 고도화 하여 서로 성능을 비교하고, 가장 우수한 모델에 선술한 정제 데이터를 연결하는 것이 1차 목표이며, 이후에 하이퍼-파라미터를 조정하며 성능 변화 확인을 시도했다. 실험은 기본 모델에서 한 번에 하나의 변수를 변경하는 방식으로 설계하였으며, 그 내용을 [표 5-2]로 정리하였다.

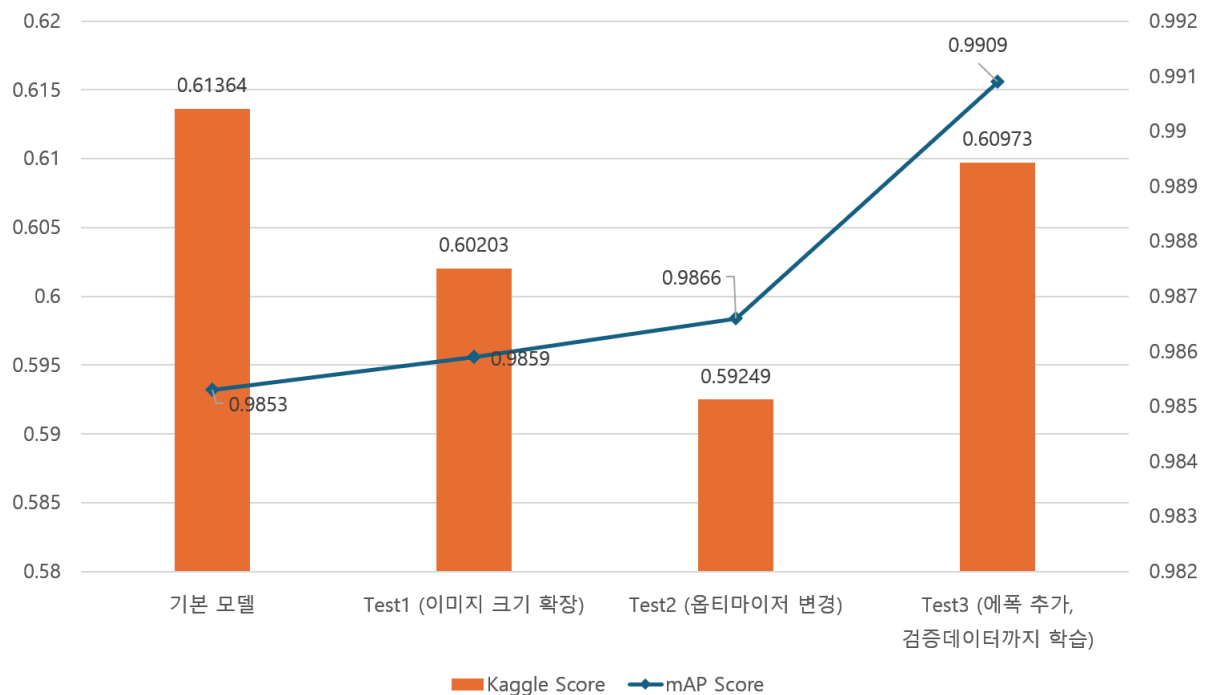
[표 5-2] 기초모델 및 실험모델의 개요

| 실험 구분                | 튜닝 항목  |       |         |         |          |
|----------------------|--------|-------|---------|---------|----------|
|                      | 이미지 크기 | 옵티마이저 | 학습데이터 수 | Epoch 수 | Batch 크기 |
| 기본 모델(대조군)           | 640    | AdamW | 6,435   | 50      | 16       |
| Test1 (이미지 크기 확장)    | 960    | AdamW | 6,435   | 50      | 8        |
| Test2 (옵티마이저 변경)     | 640    | SGD   | 6,435   | 50      | 16       |
| Test3 (Epoch+, 데이터+) | 640    | AdamW | 7,147   | 100     | 16       |

이 실험의 대조군인 기본 모델은 YOLO v11n 사전학습 가중치를 기반으로 이미지 크기 640, 배치 크기 16, 50 epoch, 옵티마이저(자동 설정)으로 학습하였으며, Kaggle 점수는 0.61364를 받았다. 이는 기본 모델 선발 당시의 점수인 0.61139와 유사한 수준으로 외부 데이터 전체 통합과 매칭 오류 정제를 거친 이후에도 성능이 크게 개선되지 않았다.

### 5.2.2. 실험 결과

세 실험 모두 대조군 대비 성능이 향상되지 않았다. 이미지를 키운 **Test1**의 경우 **Kaggle Score**가 **0.60203**으로 소폭 하락하였으며, 원본 해상도에 가까운 크기로 학습하면 경계 상자 정밀도가 향상될 것이라는 가설은 기각되었다. 기본 모델에서 옵티마이저를 변경한 **Test4**는 가장 낮은 점수를 받았다. 모델 성능에 긍정적인 하이퍼-파라미터를 찾지 못하여, 기본 모델을 추가적인 학습을 실행한 경우 어떤 변화가 이뤄지는지를 **Test3**에서 확인했다. **Epoch**을 기존 **50**에서 **100**회를 늘리고, 검증데이터까지 학습에 포함시켜 최대한 학습량을 늘려 성능을 개선하고자 했으나, 점수가 **0.60973**으로 높지 않았다. 각 실험의 결과는 [그림 5-3]와 같다.



[그림 5-3] 실험 모델별 mAP 및 Kaggle Score

### 5.2.3. 평가

기본 모델과 실험 모델들 모두 **mAP**가 **0.98** 이상으로 나타났다. 그 차이도 **0.2%** 정도로 아주 작은 차이를 가진다. 이는 검증 데이터가 학습 데이터와 동일한 촬영 환경 및 알약 구성을 공유하고 있어 검증 성능이 포화 상태에 이르렀기 때문으로 판단되며, **mAP** 점수만으로 모델간의 성능 평가를 할 수 없다. [그림 5-3]을 통하여 **mAP** 점수와 **Kaggle** 점수의 관련성이 없음 볼 수 있다.

이러한 결과가 나타난 원인으로는 학습 데이터의 대다수가 **Ai Hub** 외부 데이터로 구성되어 있는 반면 **Kaggle** 테스트 데이터는 대회 원본 특정 클래스에 집중되어 있을 가능성을 배제할 수 없다고 생각한다. 외부 데이터 도입 시기가 늦어져 해당 주장에 대한 근거 수집과 추가적인 실험을 진행할 수 없었다.

## 6. 향후 개선방향

- 더 큰 모델 사용: 컴퓨팅 자원의 부족으로 **RF-DETR Nano**만 사용했지만 **Small/Medium**은 **config**만 바꿔 실험할 수 있다.
- 라벨링 수정: 라벨링 오류로 학습에서 제외했던 데이터들의 라벨링을 수정한다.
- 데이터 전처리 통일: 직접 전수 검사하고 기록하신 파일이 **sorted** 순서대로 부여한 **uid**를 기록하고 있기 때문에 다른 팀원이 바로 사용할 수 없었다. 다른 환경에서도 바로 사용할 수 있도록 수정한다.